

A number of aliases are available for the hour, minute, second, year, month and day format string constructors. A few examples are given below:

```
(ts-hour (ts-now))
18
(ts-H (ts-now))
18

(ts-minute (ts-now))
46
(ts-min (ts-now))
46
(ts-M (ts-now))
46

(ts-second (ts-now))
16
(ts-sec (ts-now))
16
(ts-S (ts-now))
16

(ts-year (ts-now))
2019
(ts-Y (ts-now))
2019

(ts-month (ts-now))
7
(ts-m (ts-now))
7

(ts-day (ts-now))
5
(ts-d (ts-now))
5
```

You can parse a string into a timestamp object using the *ts-parse* function. For example:

```
(ts-format nil (ts-parse "Fri Dec 6 2019 18:48:00"))
"2019-12-06 18:48:00 +0530"
```

You can also format the difference between two timestamps in a human readable format as shown in the following example:

```
(ts-human-format-duration
  (ts-difference (ts-now)
    (ts-adjust 'day -1 'hour -3 'minute -2
      'second -4 (ts-now))))
"1 days, 3 hours, 2 minutes, 4 seconds"
```

The timestamp comparator operations are also defined in *ts.el*. The *ts<* function compares if one epoch UNIX timestamp is less than the other. Its definition is as follows:

```
(defun ts< (a b)
  "Return non-nil if timestamp A is less than timestamp B."
  (< (ts-unix a) (ts-unix b)))
```

In the example given below, the current timestamp is not less than the previous day and hence it returns nil.

```
(ts< (ts-now) (ts-adjust 'day -1 (ts-now)))
nil
```

Similarly, we have other comparator functions like *ts>*, *ts=*, *ts>=* and *ts<=*. A few examples of these function use cases are given below:

```
(ts> (ts-now) (ts-adjust 'day -1 (ts-now)))
t

(ts= (ts-now) (ts-now))
nil

(ts>= (ts-now) (ts-adjust 'day -1 (ts-now)))
t

(ts<= (ts-now) (ts-adjust 'day -2 (ts-now)))
nil
```

Benchmarking

A few performance tests can be conducted to compare the Emacs internal time values versus the UNIX timestamps. The benchmarking tests can be executed by including the *bench-multi* macro and *bench-multi-process-results* function available from <https://github.com/alphapapa/emacs-package-dev-handbook> in your Emacs environment.

You will also need to load the *dash-functional* library to use the *-on* function.

```
(require 'dash-functional)
```

The following tests have been executed on an Intel(R) Core(TM) i7-3740QM CPU at 2.70GHz with eight cores, 16GB RAM and running Ubuntu 18.04 LTS.

Formatting

The first benchmarking exercise is to compare the formatting of the UNIX timestamp and the Emacs internal time. The Emacs Lisp code to run the test is shown below:

```
(let ((format "%Y-%m-%d %H:%M:%S"))
  (bench-multi :times 100000
    :forms (("Unix timestamp" (format-time-string format
```